

Università degli Studi di Padova

AKME GROUP

Valutazione dei Capitolati

Tipo Documento	Documento interno
Stato	Approvato
Versione	1.0.0
Distribuzione	Akme Group Prof. Cardin Riccardo Prof. Vardanega Tullio

Gruppo: Akme Group
Email: akmeswe@gmail.com

Registro delle modifiche

Ver.	Data	Autore	Verificatore	Descrizione
1.0.0	28/10/2026	Davide Loparco	-	Revisione con correzioni e approvazione finale del documento
0.3.0	27/10/2026	Mattia Cadamuro	Roberto Piovesan, Davide Loparco	Aggiunta valutazione dei capitolati C6, C7 e C9
0.2.0	26/10/2026	Roberto Piovesan	Davide Loparco	Valutazione dei capitolati C3, C4 e C5
0.1.0	25/10/2026	Marco Giacon	Roberto Piovesan	Creazione e prima stesura documento

Indice

1 Introduzione **5**

2 Capitolato Selezionato: C2 - Concierge **5**
2.1 Descrizione 5
2.2 Dominio 5
2.3 Motivazioni della scelta 5
2.4 Conclusione 6

3 Capitolati di Interesse **6**
3.1 Capitolato C0 – AIRA 6
3.1.1 Descrizione 6
3.1.2 Dominio 6
3.1.3 Punti di forza 7
3.1.4 Criticità riscontrate 7
3.1.5 Conclusioni 7
3.2 Capitolato C8 – Sonar 7
3.2.1 Descrizione 7
3.2.2 Dominio 8
3.2.3 Punti di forza 8
3.2.4 Criticità riscontrate 9
3.2.5 Conclusioni 9

4 Altri Capitolati Valutati **9**
4.1 Capitolato C1 – AURA 9
4.1.1 Descrizione 9
4.1.2 Punti di forza 10
4.1.3 Criticità riscontrate 10
4.2 Capitolato C3 – FossDot 11
4.2.1 Descrizione 11
4.2.2 Punti di forza 11
4.2.3 Criticità riscontrate 11
4.3 Capitolato C4 – GVSPV 12
4.3.1 Descrizione 12
4.3.2 Punti di forza 12
4.3.3 Criticità riscontrate 13
4.4 Capitolato C5 – RareCare 13
4.4.1 Descrizione 13
4.4.2 Punti di forza 13
4.4.3 Criticità riscontrate 14
4.5 Capitolato C6 – Caching Semantico e Dashboard FinOps 14
4.5.1 Descrizione 14
4.5.2 Punti di forza 15
4.5.3 Criticità riscontrate 15
4.6 Capitolato C7 – SmartDashboard 15
4.6.1 Descrizione 15

4.6.2 Punti di forza 16
4.6.3 Criticità riscontrate 16
4.7 Capitolato C9 – SpecCheck 16
4.7.1 Descrizione 16
4.7.2 Punti di forza 17
4.7.3 Criticità riscontrate 17

1 Introduzione

Nel presente documento viene esposta l'analisi dei capitolati d'appalto esaminati dal gruppo **Akme Group**, al fine di individuare quello più adatto alle competenze e agli obiettivi del team.

Vengono illustrate le motivazioni che hanno portato alla candidatura per il capitolato **C2 - Concierge**, proposto da *Eggon*, insieme alle altre proposte prese in considerazione durante la valutazione.

2 Capitolato Selezionato: C2 - Concierge

2.1 Descrizione

Proponente: Eggon s.r.l.

Il capitolato C2 richiede lo sviluppo di **NEXUM Concierge**, un assistente conversazionale[G] da inserire nella PWA[G] della piattaforma NEXUM, dedicata alla comunicazione aziendale e alla gestione dei servizi HR. Tramite una chat testuale, il personale può trovare informazioni, comprendere procedure e accedere ai servizi aziendali usando il linguaggio naturale.

Il Concierge collega la conoscenza aziendale alle funzioni operative della piattaforma. L'MVP[G] è limitato all'area **ferie e permessi** e deve assicurare quattro funzionalità:

- rispondere a domande sui regolamenti aziendali, usando fonti caricate dal backoffice[G]
- spiegare l'uso delle funzioni di NEXUM sulla base dei manuali del prodotto;
- recuperare e presentare i dati personali su ferie e permessi;
- preparare e registrare una richiesta, dopo riepilogo e conferma esplicita della persona.

Il progetto comprende anche un backoffice per caricare, organizzare, sostituire e ritirare le fonti, oltre a autorizzazioni, gestione degli errori e tracciabilità di fonti ed esiti.

2.2 Dominio

Il dominio è quello della gestione HR e della comunicazione aziendale, in particolare ferie e permessi. Dal punto di vista tecnico il progetto riguarda:

- **LLM[G]** e recupero della conoscenza (**RAG[G]**), con citazione delle fonti e astensione quando le informazioni non bastano;
- elaborazione di documenti PDF in modo asincrono;
- integrazione con servizi applicativi tramite **gate** e abilità, con servizi NEXUM di prova o simulatori;
- una PWA come interfaccia per il personale e un backoffice per l'amministrazione.

Stack, modelli e servizi vengono concordati con Eggon.

2.3 Motivazioni della scelta

- **Tecnologie attuali:** il progetto permette di lavorare con modelli linguistici, recupero della conoscenza e integrazione di strumenti, ambiti molto richiesti.
- **Perimetro definito:** l'MVP riguarda solo ferie e permessi, con criteri di accettazione e prove descritti in modo esplicito.
- **Supporto del proponente:** sono previsti incontri periodici, un gruppo Telegram dedicato, un referente Eggon e la fornitura di manuali, regolamenti e dati di prova.

- **Libertà nelle scelte tecniche:** il team può proporre tecnologie e architettura, motivandole rispetto ai risultati.
- **Utilità concreta:** il prodotto risolve un problema reale, ridurre le richieste ripetitive e semplificare l'accesso ai servizi HR.

2.4 Conclusione

Sintesi finale: il capitolato C2 offre un buon equilibrio tra novità tecnologica e ambito circoscritto, ed è quello più coerente con le competenze e gli obiettivi del gruppo. Le principali criticità, come l'ampiezza delle quattro capacità da integrare e la dipendenza dalle API di NEXUM, sono considerate gestibili grazie al percorso in tre fasi e al supporto di Eggon.

3 Capitolati di Interesse

3.1 Capitolato C0 – AIRA

3.1.1 Descrizione

Proponente: San Marco SPA

Il capitolato C0 propone lo sviluppo di **AIRA** (AI Requests Automation), un tool multimodale basato su Intelligenza Artificiale che riceve richieste da canali eterogenei (e-mail con allegati, note vocali WhatsApp[G], immagini, cartelle del filesystem), ne estrae il significato e le trasforma in dati strutturati, come JSON[G] o record di database. Le informazioni vengono poi inoltrate in automatico ai sistemi aziendali tramite API REST[G], oppure notificate via e-mail, eliminando i passaggi manuali.

Le funzionalità richieste comprendono il monitoraggio continuo dei canali, un filtro sui dati in ingresso per evitare accessi non autorizzati o commistione di informazioni, l'elaborazione semantica con LLM, l'avvio automatico di azioni e un'interfaccia amministrativa per configurazione, log e persistenza dei dati. Sono opzionali una dashboard analitica e l'invio autonomo di risposte al mittente.

3.1.2 Dominio

Il dominio è l'automazione dei processi commerciali e di assistenza in ambito Business-to-Business: inserimento di ordini d'acquisto, apertura di ticket, prenotazione di appuntamenti, segnalazione di opportunità e richiesta di documenti contabili. Dal punto di vista tecnico il progetto riguarda:

- acquisizione multimodale: speech-to-text per l'audio e OCR[G] / visione artificiale per immagini e documenti scansionati;
- elaborazione semantica con LLM, con identificazione di entità e intenzioni;
- integrazione con applicativi esterni tramite API REST;
- un'interfaccia web amministrativa e un database relazionale interno.

Le tecnologie consigliate sono Java[G] con Spring Boot, LangChain4j[G], Angular[G] e TypeScript[G], Whisper[G], Tesseract[G] ed EasyOCR[G], con persistenza su SQLite[G], MariaDB o PostgreSQL[G].

3.1.3 Punti di forza

- **Tecnologie attuali e varie:** il progetto unisce LLM, speech-to-text, OCR e visione artificiale, un insieme di competenze molto spendibili.
- **Ambito concreto e comprensibile:** automatizzare l'inserimento di ordini e richieste è un caso d'uso chiaro, con benefici evidenti per l'azienda.
- **Stack consigliato e architettura indicata:** il capitolato propone strumenti consolidati (Java/ Spring Boot, Angular) e uno schema architetturale di partenza.
- **Supporto del proponente:** San Marco affianca il gruppo con figure dedicate per supporto tecnico e setup degli ambienti, con metodologia Agile e referenti indicati.
- **Struttura modulare:** è richiesta un'architettura che permetta di aggiungere in futuro nuovi canali e funzionalità.

3.1.4 Criticità riscontrate

- **Molti canali e formati da gestire:** e-mail, WhatsApp, immagini, audio e filesystem richiedono tecnologie diverse, con il rischio di disperdere il lavoro su troppi fronti.
- **Qualità e affidabilità dell'estrazione:** l'OCR su foto di cataloghi e il riconoscimento vocale possono dare errori, e la mappatura sui codici articolo ufficiali richiede una validazione accurata.
- **Dati sensibili e isolamento:** è richiesto un filtro di sicurezza con identificazione dei tenant, che aggiunge complessità progettuale.
- **Scalabilità richiesta:** l'architettura deve sostenere elevati volumi di ordini, e la pipeline deve essere coperta da test di unità e di integrazione.
- **Integrazione con sistemi esterni:** ERP[G], CRM[G] e portali B2B richiedono API di cui il gruppo dipende.

3.1.5 Conclusioni

Il capitolato C0 è stato valutato come la nostra seconda scelta: offre un ambito attuale e concreto, con un buon supporto aziendale, ma abbiamo giudicato più equilibrato il capitolato C2 – Concierge.

3.2 Capitolato C8 – Sonar

3.2.1 Descrizione

Proponente: Halue S.r.l. Società Benefit

Il capitolato C8 richiede lo sviluppo di **Sonar**, un'applicazione web che si collega a uno store Shopify[G] di grandi dimensioni, ne interroga sistematicamente configurazioni e codice, e ricostruisce la mappa di cosa esiste, di cosa non serve più e delle eventuali dipendenze. Il problema nasce dalla stratificazione degli store nel tempo: app dimenticate ma ancora fatturate, webhook[G] verso fornitori non più attivi, campi personalizzati mai letti e codice di tema non più referenziato, legati a una conoscenza storica del sistema ormai andata persa.

Il sistema deve produrre due risultati: una **documentazione sempre aggiornata** dello store, consultabile via web ed esportabile in Markdown[G] o HTML[G], e un **rapporto di igiene tecnica** che segnala orfani e codice morto, con una motivazione esplicita per ogni segnalazione. Le funzioni

principali sono la scansione durevole dello store, la costruzione di un modello a grafo delle entità e delle dipendenze, l'analisi statica del tema, un motore di regole estensibile, il confronto tra snapshot e l'esportazione della documentazione. Sonar è di sola lettura e non modifica lo store.

3.2.2 Dominio

Il dominio è l'e-commerce su piattaforma Shopify (Shopify Plus) e in particolare la documentazione e la manutenzione di store enterprise: passaggi di consegne tra agenzie, onboarding di nuovi team, audit[G] di spesa sulle app e valutazione di migrazioni (replatforming[G]). Dal punto di vista tecnico il progetto riguarda:

- acquisizione dati dalle Admin GraphQL API[G] di Shopify, con gestione del rate limiting[G] e scansioni lunghe, ripartibili e osservabili;
- normalizzazione dei dati in un grafo di entità e dipendenze, con fonte dell'evidenza e livello di confidenza (accertato, probabile, ipotizzato);
- analisi statica del codice di tema (Liquid[G], JSON, JavaScript[G], CSS[G]);

3.2.2.0.0.0.1

Il dominio è l'e-commerce su piattaforma Shopify (Shopify Plus) e in particolare la documentazione e la manutenzione di store enterprise: passaggi di consegne tra agenzie, onboarding di nuovi team, audit di spesa sulle app e valutazione di migrazioni. Dal punto di vista tecnico il progetto riguarda:

- acquisizione dati dalle Admin GraphQL[G] API di Shopify, con gestione del rate limiting e scansioni lunghe, ripartibili e osservabili;
- normalizzazione dei dati in un grafo di entità e dipendenze, con fonte dell'evidenza e livello di confidenza;
- analisi statica del codice di tema (Liquid, JSON, JavaScript, CSS);
- campionamento statistico per misurare l'uso reale dei campi personalizzati;
- tecnologie vincolanti: Next.js[G] e Vercel, Supabase[G] e TypeScript.

3.2.3 Punti di forza

- **Problema reale e ben motivato:** il capitolato descrive in modo chiaro una necessità concreta di Halue, con costi e scenari d'uso precisi.
- **Requisiti e criteri di accettazione molto dettagliati:** obbligatori, desiderabili e opzionali sono elencati con identificativi, e i criteri di accettazione sono misurabili.
- **Spazio di negoziazione:** l'elenco dei requisiti obbligatori può essere ridotto con una proposta motivata, concordata per iscritto con Halue.
- **Architettura aperta:** quella proposta è solo un'ipotesi non vincolante; il gruppo può proporne una diversa se motivata.
- **Uso dell'AI incoraggiato:** gli strumenti di sviluppo basati su AI sono previsti e la complessità è calibrata assumendo che vengano usati.
- **Supporto e presidio dei consumi:** sono previsti incontri ogni due settimane, una casella di posta dedicata con risposta entro un giorno lavorativo e un controllo periodico dei limiti dei piani gratuiti.

- **Competenze spendibili:** grafi, API reali, esecuzione durevole e analisi statica sono temi formativi di alto livello.

3.2.4 Criticità riscontrate

- **Stack tecnologico vincolante:** Next.js su Vercel, Vercel Workflows[G], Supabase e TypeScript sono obbligatori; richiedere una deroga è possibile solo entro la fase di analisi dei requisiti.
- **Molti requisiti obbligatori:** sono previsti venti requisiti funzionali obbligatori, sette di qualità e sicurezza e sette non funzionali, e l'ambito è ampio rispetto al tempo disponibile.
- **Criteri di accettazione stringenti:** richiedono almeno il 95% di test superati, copertura di almeno il 70%, tasso di falsi positivi sulle segnalazioni ad alta severità non superiore al 10% e una documentazione giudicata utilizzabile in un passaggio di consegne reale.
- **Dominio Shopify da apprendere:** modello a punti delle API, metafield, blocchi app e Shopify Functions richiedono tempo di studio prima di poter progettare.
- **Ambiente di lavoro da costruire:** Halue non fornisce store, dati né infrastruttura; il gruppo deve creare e popolare da sé uno store di sviluppo con una stratificazione realistica.
- **Limiti dei piani gratuiti:** Supabase sospende i progetti inattivi e Vercel impone soglie mensili ridotte su Workflows, Blob, Sandbox e AI Gateway[G], il che incide sulle ri-scansioni pianificate.
- **Nessun incontro in presenza e nessun canale di messaggistica condiviso:** la comunicazione è affidata a videoconferenze e a una casella di posta.

3.2.5 Conclusioni

Il capitolato C8 è stato valutato come la nostra terza scelta: è ben strutturato e offre un buon margine di negoziazione, ma lo stack vincolante, l'ampiezza dei requisiti obbligatori e la necessità di costruire autonomamente l'ambiente di lavoro sono stati determinanti nella nostra decisione di preferenza.

4 Altri Capitolati Valutati

4.1 Capitolato C1 – AURA

4.1.1 Descrizione

Proponente: Vimar S.p.A.

Il capitolato C1, proposto dall'Area Ricerca e Sviluppo Elettronica di Vimar, richiede di progettare e realizzare **AURA** (*AI for Understanding, Reasoning and Automation*), un dimostratore IoT + AI per il settore residenziale. Il cuore del progetto è un **server MCP[G]** (Model Context Protocol) e un client MCP che rendono le KNX IoT 3rd party API[G] di Vimar utilizzabili da servizi di intelligenza artificiale generativa commerciali, come Claude[G], ChatGPT[G], Copilot[G] e Gemini[G]. In questo modo l'utente può interrogare e comandare il proprio impianto domotico View Wireless in linguaggio naturale, autenticandosi con il proprio account MyVimar.

Il progetto comprende tre componenti obbligatori: un'infrastruttura Cloud-ready[G] basata su container[G] e Infrastructure as Code[G], il server e il client MCP, e il collegamento con un agente[G] o modello AI esterno che gestisca casi d'uso base e avanzati (selezione dell'impianto, lettura e scrittura dello stato dei dispositivi, richieste ambigue o implicite, conferme per comandi estesi).

Sono previsti come opzionali il supporto al multi-impianto, allo storico, agli scenari composti e ai trigger[G].

4.1.2 Punti di forza

- **Tecnologia attuale e di interesse:** il protocollo MCP e l'integrazione con LLM commerciali sono un ambito molto recente, con competenze spendibili sul mercato.
- **Kit fisico fornito da Vimar:** ogni gruppo riceve un kit portatile con gateway, attuatori per luci e porta, comando tapparelle, sensori di presenza, termostato e pulsante allarme, oltre all'accesso alle API KNX IoT.
- **Supporto del proponente:** sono previsti SAL[G] bisettimanali, referenti Vimar contattabili via e-mail e incontri di approfondimento su Docker, KNX IoT, MCP e sviluppo con strumenti di AI.
- **Risorse Cloud messe a disposizione:** Vimar fornisce, se richiesto, servizi e spazio su AWS[G], compresi i servizi AI e LLM.
- **Libertà sullo stack:** i vincoli tecnologici sono pochi (Docker, KNX IoT, OAuth2[G], Git[G] e licenza open source); è consigliato Python[G] con FastMCP[G], ma sono ammessi anche TypeScript e Java.
- **Casi d'uso chiari:** il capitolato elenca esempi concreti di interazione, base e avanzati, con le risposte attese, e distingue requisiti obbligatori, opzionali e nice-to-have.
- **Nessuna interfaccia AI da sviluppare:** l'assistente è un servizio esterno già esistente, quindi il lavoro si concentra sull'integrazione e non sulla costruzione di un chatbot.

4.1.3 Criticità riscontrate

- **Dipendenza da servizi e hardware esterni:** il progetto si appoggia al kit fisico, alle API KNX IoT e ai servizi AI di terzi; limiti, indisponibilità o cambiamenti di questi servizi sono fuori dal controllo del gruppo.
- **Accesso alla documentazione soggetto a registrazione:** la documentazione KNX IoT è scaricabile solo dopo la registrazione sul sito Vimar e la firma di un documento d'uso.
- **Comportamento dell'AI non completamente controllabile:** i casi d'uso dipendono da come il modello esterno interpreta richieste ambigue o implicite, e rendere i risultati verificabili con i test non è semplice.
- **Vincolo sull'aggiornamento tramite Subscription:** il polling[G] non è ammesso, quindi le notifiche di impianto vanno gestite con il meccanismo push dello standard, più complesso da realizzare e da testare.
- **Requisiti di test elevati:** sono richiesti test di unità, integrazione ed end-to-end, con copertura minima del 75% per unità e integrazione e dell'80% per gli end-to-end.
- **Molti prodotti attesi:** oltre al codice sono richiesti documentazione con modello C4[G], analisi tecnica con stima dei costi AWS, documentazione del server e del client, manuale di installazione, video dimostrativo e report dei test.
- **Ambito meno incentrato sull'AI:** la parte di intelligenza artificiale è affidata a servizi esterni commerciali, quindi il gruppo lavora più sull'integrazione che sull'AI stessa.
- **Incontri in presenza obbligatori:** sono richiesti almeno due incontri in presenza presso la sede di Padova.

4.2 Capitolato C3 – FossDot

4.2.1 Descrizione

Proponente: Uranion EMEA Ltd

Il capitolato C3, proposto da Uranion EMEA Ltd, richiede di progettare e realizzare Fossdot, uno smart speaker vocale open source, estensibile e orientato alla privacy. L'obiettivo principale è superare i limiti dei dispositivi commerciali tradizionali, liberando l'utente da cloud proprietari opachi e vincoli di lock-in, consentendo la libera scelta, la configurazione del provider di intelligenza artificiale (in cloud o locale) e la cooperazione tra più dispositivi domestici.

L'infrastruttura richiesta comprende quattro componenti chiave: la selezione e validazione dell'hardware di riferimento con distinta base, il firmware embedded per la gestione audio e il rilevamento della wake word[G], il servizio server di bridging con pipeline STT[G] –> LLM –> TTS[G] e integrazione MCP, e un'applicazione web per la registrazione e gestione dei dispositivi. Il capitolato distingue una consegna minima con la dimostrazione del flusso end-to-end su hardware fisico e un unico provider, e una consegna completa estesa a più provider, agenti/skills vocali e opzioni di privacy interamente locali.

4.2.2 Punti di forza

- **Adozione di tecnologie emergenti e di interesse:** l'integrazione del MCP e l'impiego di framework moderni (come Rust[G] embedded, Embassy[G] e Axum[G]) offrono un bagaglio di competenze innovative e spendibili sul mercato.
- **Proprietà e valorizzazione Open Source:** l'intero progetto è rilasciato con licenza permissiva (Apache 2.0 o MIT) e la proprietà intellettuale del codice rimane al gruppo.
- **Indipendenza tecnologica e assenza di lock-in:** la struttura del servizio di bridging permette di alternare liberamente modelli LLM in cloud e soluzioni locali on-premise (es. Ollama).
- **Attenzione alla privacy e conformità GDPR[G]:** il sistema affronta la riservatezza delle conversazioni permettendo, come evoluzione, l'esecuzione di STT e TTS interamente in locale senza trasmettere audio verso server esterni.
- **Supporto tecnico e formazione specializzata:** Uranion mette a disposizione SAL bisettimanali e sessioni di approfondimento («tech dive») con software architect, estesi anche a metodologie di agentic coding.
- **Ampia letteratura e prior art di riferimento:** il capitolato suggerisce progetti e strumenti open source consolidati a cui ispirarsi (es. Willow[G], Rhasspy[G], openWakeWord[G], Silero VAD[G], faster-whisper, Piper[G]).
- **Ripartizione chiara tra MVP e prodotto completo:** la distinzione tra requisiti minimi dimostrabili ed estensioni evolutive garantisce una gestione graduale e controllata dell'ambito di progetto.

4.2.3 Criticità riscontrate

- **Stack tecnologico molto ampio ed eterogeneo:** il progetto copre l'intero stack architetturale (hardware, firmware embedded, server di bridging, app web e AI generativa), richiedendo al gruppo competenze multidisciplinari estese.

- **Complessità dello sviluppo su hardware fisico:** la gestione dell'audio in tempo reale, lo streaming dal microcontrollore e il rilevamento della wake word comportano sfide sistemistiche impegnative e un testing su hardware più complesso da automatizzare.
- **Nessun kit fisico o dataset fornito dal proponente:** a differenza di altri capitolati, Uranion non fornisce hardware pre-assemblato o dataset proprietari; la selezione dei componenti e l'approvvigionamento spettano al gruppo.
- **Gestione della latenza nella pipeline vocale:** l'integrazione di più fasi sequenziali richiede un'accurata ottimizzazione per garantire tempi di risposta accettabili nell'interazione vocale.
- **Curva di apprendimento per Rust in ambito embedded:** l'impiego di Rust per il firmware, pur costituendo un punto di forza, può richiedere uno sforzo iniziale significativo per chi non ha esperienza specifica.
- **Requisiti di test e documentazione elevati:** per il backend e l'interfaccia web è richiesta una copertura di test superiore all'80% corredata da report dedicati e documentazione delle scelte architetturali.

4.3 Capitolato C4 – GVSPV

4.3.1 Descrizione

Proponente: M31 S.r.l.

Il capitolato C4, proposto da M31 S.r.l., richiede di progettare e realizzare una piattaforma software distribuita per la gestione dei veicoli e dei servizi post-vendita (GVSPV) nell'ambito automotive. Il sistema mira a centralizzare e coordinare i flussi informativi tra tre attori principali: la casa automobilistica, le concessionarie o centri di assistenza e i proprietari finali dei veicoli.

L'architettura deve supportare la gestione dei modelli e dei singoli veicoli, la registrazione delle vendite, le richieste di manutenzione e appuntamenti, le campagne di richiamo e l'invio di notifiche evento-dipendenti, combinando comunicazioni sincrone via API e comunicazioni asincrone a eventi. Il progetto non richiede l'integrazione di hardware fisici o centraline reali, basandosi su dati veicolari simulati, mentre il percorso di rilascio prevede lo sviluppo graduale di un'analisi dello stato dell'arte, un Proof of Concept (PoC[G]) ed un Minimum Viable Product (MVP) basato su servizi containerizzati.

4.3.2 Punti di forza

- **Architettura distribuita di livello enterprise:** la combinazione di microservizi, comunicazioni event-driven (es. NATS[G] o Kafka[G]) e orchestrazione con Kubernetes[G] offre una solida formazione su pattern architetturali altamente richiesti nel mercato industriale.
- **Assenza di dipendenze hardware fisiche:** lo sviluppo si basa interamente su software e dati veicolari simulati, eliminando i rischi di ritardi o colli di bottiglia legati all'approvvigionamento o alla gestione di componenti fisici.
- **Stack tecnologico moderno e ben definito:** il capitolato suggerisce un ecosistema di riferimento chiaro ma non vincolante, lasciando al gruppo la flessibilità di motivare eventuali alternative.

- **Chiara articolazione delle responsabilità e dei domini:** la separazione nei tre livelli logici (casa madre, concessionaria, cliente) facilita la suddivisione dei compiti all'interno del gruppo di lavoro.

4.3.3 Criticità riscontrate

- **Elevata complessità architetturale e sistemistica:** la gestione di un'architettura distribuita a microservizi con comunicazione mista sincrona/asincrona e deployment orchestrato su Kubernetes richiede forti competenze di ingegneria del software e DevOps.
- **Molteplicità delle interfacce utente:** la piattaforma richiede lo sviluppo di tre interfacce web distinte (Single Page Application in Angular) per i tre diversi ruoli di utente, comportando un notevole carico di lavoro sul fronte front-end.
- **Rigorosi requisiti di sicurezza e segregazione dei dati:** il sistema deve garantire un controllo degli accessi basato sui ruoli (RBAC[G]) e l'isolamento completo dei dati tra differenti concessionarie e proprietari.
- **Elevati standard di test e validazione:** M31 richiede la stesura di un «Test Book» esaustivo e l'implementazione di test unitari, di integrazione, end-to-end, di segregazione dei dati e di scalabilità sotto carico.
- **Significativo onere documentale:** oltre al codice, è richiesta una documentazione strutturata e approfondita con diagrammi architetturali (C4 Model e Mermaid[G]), documentazione API, manuale dei test e manuali utente/amministratore.

4.4 Capitolato C5 – RareCare

4.4.1 Descrizione

Proponente: Miriade S.r.l.

Il capitolato C5, proposto da Miriade S.r.l., richiede di progettare e realizzare RareCare Navigator, una piattaforma digitale e assistente virtuale basato su intelligenza artificiale per il supporto alla gestione delle malattie rare e complesse, focalizzata inizialmente sulla Sindrome di Sanfilippo (MPS III). Il sistema mira ad abbattere l'isolamento delle famiglie e la frammentazione dei servizi affiancando la figura del Care Navigator e connettendo in sicurezza centri clinici, medici, terapisti, strutture scolastiche ed enti amministrativi.

La piattaforma integra un assistente conversazionale basato su architettura RAG per l'interpretazione di protocolli e pratiche burocratiche basato su fonti ufficiali, un hub di coordinamento multilivello con permessi granulari, un motore per la gestione di scadenze e checklist di cura dinamiche e un archivio documentale cifrato. L'architettura è concepita secondo un modello modulare a «Disease Pack» (plugin per estendere il sistema ad altre patologie senza modificare il core), raccomandando un'infrastruttura cloud-native serverless (AWS Lambda[G], Bedrock[G], API Gateway) e un'applicazione mobile/web sviluppata in Flutter[G] o React Native[G].

4.4.2 Punti di forza

- **Elevato impatto sociale e applicazione reale dell'AI:** il progetto applica soluzioni di GenAI[G], RAG e NLP[G] a un contesto sanitario ed etico di forte rilievo, in diretta collaborazione con l'associazione Sanfilippo Fighters e con validazione sul campo.

- **Architettura modulare basata su «Disease Pack»:** la netta separazione tra la logica generale della piattaforma e i moduli specifici per patologia rende il sistema altamente riusabile ed estendibile senza dover riscrivere il codice base.
- **Stack tecnologico serverless e cross-platform professionale:** l'impiego di tecnologie cloud-native su AWS e di framework come Flutter o React[G] Native consente di acquisire competenze di alto livello ed estremamente spendibili sul mercato.
- **Integrazione di GenAI affidabile tramite RAG e guardrail:** l'uso del Retrieval-Augmented Generation e di filtri di sicurezza garantisce che l'assistente risponda basandosi esclusivamente su fonti verificate, minimizzando il rischio di allucinazioni.
- **Eccellente supporto, mentorship e formazione del proponente:** Miriade garantisce referenti dedicati per architettura cloud, sviluppo full-stack/AI e UX/UI, oltre a una formazione iniziale specifica sul dominio clinico e sulla privacy dei dati sanitari.
- **Flessibilità nella definizione dell'ambito di progetto:** Miriade si dichiara disponibile a discutere e concordare con il gruppo un perimetro di lavoro proporzionato ed equilibrato rispetto alle risorse disponibili.

4.4.3 Criticità riscontrate

- **Elevati vincoli di sicurezza, privacy e conformità GDPR:** trattando dati sanitari appartenenti a categorie particolari (art. 9 GDPR) relativi a soggetti fragili o minori, il sistema impone la cifratura dei dati in transito e a riposo, autenticazione forte (MFA[G]), tracciamento degli accessi (audit) e una valutazione d'impatto (DPIA[G]).
- **Gestione complessa dei permessi e degli accessi:** il controllo basato sui soli ruoli (RBAC) risulta insufficiente, richiedendo la modellazione di regole basate anche sulla relazione specifica con il paziente e sul contesto.
- **Responsabilità etico-clinica e validazione dell'AI:** l'impiego di modelli generativi in ambito sanitario richiede l'introduzione di meccanismi di revisione umana («human-in-the-loop») per la conferma dei dati estratti.
- **Adattamento delle pipeline AI alla lingua italiana:** alcuni servizi clinici nativi in cloud (es. Amazon Comprehend Medical[G]) sono disponibili solo in inglese, imponendo al gruppo la progettazione di pipeline alternative per l'estrazione di entità da testi in italiano.
- **Elevata articolazione funzionale e integrazione dei moduli:** la convivenza di assistente RAG, motore di workflow[G] per le scadenze, gestione documentale e comunicazioni richiede un'attenta coordinazione dei servizi cloud asincroni e degli eventi.

4.5 Capitolato C6 – Caching Semantico e Dashboard FinOps

4.5.1 Descrizione

Proponente: Var Group

Il capitolato C6, proposto da Var Group, richiede di estendere il proxy open source **LiteLLM[G]** attraverso lo sviluppo di due componenti originali volte a ottimizzare l'uso dei modelli di intelligenza artificiale generativa in contesti aziendali: un sistema di caching semantico e una dashboard FinOps.

Il primo componente implementa un livello di cache basato su embedding e ricerca per similarità semantica, in grado di riconoscere richieste equivalenti, restituire risposte pre-salvate e tracciare il risparmio di token e latenza, gestendo soglie di similarità configurabili, invalidazione tramite TTL/versioning e persistenza su Redis[G] o Vector DB. Il secondo componente consiste in un servizio separato per il tracciamento dei costi (FinOps), con aggregazione dei consumi per team, progetto e utente, calcolo del risparmio generato dalla cache, gestione di budget, avvisi e previsioni di spesa mensili con supporto al multi-tenancy[G] e aggiornamento in tempo quasi reale.

4.5.2 Punti di forza

- **Architettura di partenza consolidata:** l'adozione di LiteLLM come proxy/orchestratore fornisce una base solida già operante in produzione, evitando di reinventare componenti di routing e gestione credenziali e permettendo di focalizzarsi sulle estensioni richieste.
- **Risoluzione di problemi reali di produzione:** sia il caching semantico che la gestione dei costi (FinOps) rispondono a esigenze concrete del mercato enterprise.
- **Stack tecnologico moderno e flessibile:** la libertà di scelta per gli embedding (OpenAI, sentence-transformers[G]) e i database (Redis, Qdrant[G], pgvector[G], PostgreSQL) consente di adattare lo stack alle competenze del team.
- **Eccellente supporto e processo del proponente:** Var Group offre sessioni di design thinking, User Story Mapping, formazione tecnologica, standup periodici e review condivise.
- **Modularità del sistema:** la netta separazione tra il middleware di caching e il servizio per la dashboard FinOps facilita la suddivisione del lavoro tra i componenti del gruppo.

4.5.3 Criticità riscontrate

- **Complessità del caching semantico:** tarare la soglia di similarità coseno[G] e gestire correttamente l'invalidazione della cache (TTL e versioning dei modelli) sono compiti complessi per evitare di servire risposte obsolete o semanticamente scorrette.
- **Difficoltà nella validazione e testing dei modelli:** il vincolo di copertura test $\geq 70\%$ richiede la netta separazione tra test deterministici e valutazione probabilistica delle similarità degli embedding.
- **Gestione del multi-tenancy e isolamento dei dati:** garantire la segregazione rigorosa delle metriche e dei dati tra differenti team/progetti nella dashboard FinOps aumenta la complessità architetturale.
- **Aggregazione dei dati in quasi real-time:** la componente FinOps richiede la raccolta e l'elaborazione continua e performante degli audit log e dei token senza introdurre latenza nell'instradamento delle richieste LLM.

4.6 Capitolato C7 – SmartDashboard

4.6.1 Descrizione

Proponente: Ergon Informatica Srl

Il capitolato C7, proposto da Ergon Informatica Srl, richiede lo sviluppo di **SmartDashboard**, una piattaforma di Business Intelligence self-service potenziata dall'intelligenza artificiale, progettata per consentire agli utenti aziendali di consultare, analizzare e interpretare autonomamente i dati provenienti dagli ERP.

L'architettura di riferimento prevede un Raw Data Store per i dati grezzi, un Data Warehouse basato su un modello dimensionale (star schema) ottimizzato tramite un motore analitico OLAP[G] come DuckDB[G], e una Web App responsive dotata di dashboard builder drag & drop, libreria di widget interattivi e un assistente AI integrato. Il layer di intelligenza artificiale funge da interprete per generare visualizzazioni in linguaggio naturale, produrre insight automatici, suggerire widget e spiegare le variazioni degli indicatori, protetto da rigorosi controlli di sicurezza e guardrail.

4.6.2 Punti di forza

- **Architettura fortemente disaccoppiata:** Il capitolato propone una netta suddivisione in tre livelli (Raw Data Store, Data Warehouse, Web App). Questa impostazione facilita enormemente la stesura della documentazione architetturale in Typst[G] e rende il codice molto più manutenibile e testabile.
- **Facilità di divisione dei compiti:** Grazie all'impostazione modulare e alla comunicazione tramite API REST, il lavoro all'interno del team può essere parallelizzato con grande efficienza.
- **Approccio di sviluppo iterativo chiaro:** Il proponente suggerisce di non sviluppare tutto subito, ma di procedere per fasi (MVP della pipeline BI, poi integrazione AI di base, poi insight automatici).
- **Stack tecnologico bilanciato:** Il progetto unisce tecnologie web classiche con elementi altamente innovativi ma circoscritti, come i database orientati all'analisi e le API dei LLM.

4.6.3 Criticità riscontrate

- **Ampiezza e complessità complessiva:** la realizzazione congiunta di una pipeline ETL[G], di un data warehouse dimensionale, di un'interfaccia web self-service avanzata e di un layer AI richiede un impegno sistemistico notevole.
- **Gestione dell'ambiguità e affidabilità dell'AI:** le richieste in linguaggio naturale possono essere ambigue e presentano il rischio di generare risultati errati ma plausibili.
- **Progettazione del modello dimensionale:** la corretta modellazione dei dati aziendali in uno schema a stella richiede competenze specifiche di data warehousing per evitare inefficienze nelle aggregazioni.

4.7 Capitolato C9 – SpecCheck

4.7.1 Descrizione

Proponente: Zucchetti Academy

Il capitolato C9 richiede lo sviluppo di **SpecCheck**, un validatore automatizzato progettato per l'agente di harness *OpenCode*. Il progetto risponde alla nuova sfida dello sviluppo software guidato dall'intelligenza artificiale, in cui l'IA automatizza la scrittura del codice e sorge un nuovo collo di bottiglia: il controllo di qualità e la coerenza progettuale tra le specifiche e l'implementazione reale.

Il sistema si articola principalmente in due fasi:

- **Fase 1 (Casi d'Uso -> Specifiche):** parsing dei diagrammi e documenti Markdown per verificare che ogni caso d'uso sia tradotto correttamente nei requisiti di progetto;

- **Fase 2 (Specifiche -> Codice):** analisi comportamentale del codice sorgente da parte dell'LLM per verificare la reale implementazione delle funzionalità richieste.

Il progetto comprende inoltre la realizzazione di una dashboard integrata (eseguita via plugin) per mostrare lo stato di copertura e dei test con evidenza immediata delle discrepanze, e un installer completo per l'integrazione in OpenCode[G] tramite script di installazione automatizzati.

4.7.2 Punti di forza

- **Tematica all'avanguardia:** il progetto affronta direttamente i paradigmi più moderni dell'ingegneria del software basata su LLM, focalizzandosi sull'harness engineering[G], sulla validazione automatica e sul controllo della coerenza progettuale.
- **Chiara definizione delle fasi e degli obiettivi:** il perimetro di lavoro è suddiviso in moduli logici ben definiti (parsing dei diagrammi, analisi comportamentale, dashboard integrata e pacchetto di installazione).
- **Supporto di un proponente strutturato:** l'affiancamento da parte di Zucchetti garantisce un contesto di riferimento solido, orientato alla condivisione di best practices e metodologie di sviluppo innovative.

4.7.3 Criticità riscontrate

- **Complessità dell'analisi semantica e comportamentale:** interpretare in modo automatizzato diagrammi eterogenei (immagini, XMI[G], Mermaid) e analizzare il comportamento del codice sorgente tramite LLM richiede una gestione accurata per evitare falsi positivi o valutazioni imprecise.
- **Integrazione profonda con l'ecosistema OpenCode:** lo sviluppo di plugin, skill, comandi e tool dedicati all'estensione di OpenCode richiede un forte sforzo di comprensione architetturale della piattaforma di riferimento.
- **Gestione dei test e della robustezza del validatore:** trattandosi di un tool di controllo qualità, il sistema stesso deve garantire elevati standard di affidabilità, precisione e copertura di test.
- **Novità concettuale dell'Harness Engineering:** la curva di apprendimento iniziale per comprendere a fondo le dinamiche e i pattern dell'harness engineering e della programmazione in linguaggio naturale può richiedere un impegno di studio significativo.

Davide Loparco



Firma del revisore interno